

DM2026 Assignment 2: Cross-Validation, SVM, Association Rules, and Clustering

Yu-Shun Liu
Student ID: 413551030

GitHub Repository: <https://github.com/RaisoLiu/DM2026-Assignment-2>

1 Q1: K-Fold Cross-Validation (15 pts)

Continuation of Assignment 1 `RealWorldClassification.ipynb`, using the updated `linear_model.py` from the course GitHub. The same preprocessing pipeline is reproduced: `LabelEncoder` on Species, `KNNImputer`($k = 5$), train/test split (70/30, seed=40), and `MinMaxScaler` (fit on training data only).

1.1 1(a) 5-Fold Cross-Validation: 4×4 Hyperparameter Grid

We perform 5-fold CV (`KFold(n_splits=5, shuffle=True, random_state=40)`) on the training set (350 samples) over 16 combinations of learning rate and L2 regularization strength. The model uses `n_iteration=10000`, `val_ratio=0.2`.

Table 1: Mean CV accuracy for each (lr, λ) combination

	$\lambda = 1.0$	$\lambda = 2.0$	$\lambda = 4.0$	$\lambda = 8.0$
lr=0.005	0.7200	0.7257	0.7314	0.7257
lr=0.01	0.7257	0.7286	0.7286	0.7229
lr=0.1	0.7171	0.7286	0.7257	0.7229
lr=0.5	0.7171	0.6886	0.5229	0.4657

1.2 1(b) Top 2 Hyperparameter Settings — Test Evaluation

The top 2 settings by CV accuracy are retrained on the full training set and evaluated on the held-out test set (150 samples).

Table 2: Test evaluation for top 2 hyperparameter settings

Rank	Setting	Accuracy	Precision	Recall	F1-score
#1	lr=0.005, $\lambda=4.0$	0.7600	0.7273	0.8421	0.7805
#2	lr=0.1, $\lambda=2.0$	0.7467	0.7215	0.7895	0.7540

1.3 1(c) Observations

- **Small learning rates are robust to regularization strength.** Learning rates 0.005, 0.01, and 0.1 achieve similar CV accuracy ($\sim 72\text{--}73\%$) across all λ values, indicating that slow, stable convergence makes the model less sensitive to the regularization hyperparameter.
- **Large learning rate + strong regularization causes severe underfitting.** At $\text{lr}=0.5$, performance degrades sharply when $\lambda \geq 4.0$ (CV accuracy drops to 52.3% and 46.6%). When gradient steps are too large and regularization aggressively shrinks weights, the model cannot learn meaningful patterns.
- **Good generalization from CV to test.** The top-1 setting ($\text{lr}=0.005$, $\lambda=4.0$) achieves 73.1% CV accuracy and 76.0% test accuracy, and the top-2 setting generalizes similarly ($\sim 73\% \rightarrow 74.7\%$). This confirms that 5-fold CV on this dataset provides a reliable estimate of out-of-sample performance.
- The top-1 model shows higher recall (84.2%) than precision (72.7%), meaning it favors identifying positive samples at the cost of more false positives.

2 Q2: SVM Classification (15 pts)

We use the `mobile_price.csv` dataset (2000 samples, 20 features, 4 balanced classes). Data is split 60/20/20 into train/validation/test sets (`random_state=42`). F1-score uses `macro` averaging throughout.

2.1 2(a) SVM with $C=1.0$

Table 3: SVM ($C=1.0$) performance across data splits

Set	Accuracy	F1 (macro)
Train	0.9500	0.9505
Validation	0.9475	0.9463
Test	0.9625	0.9617

2.2 2(b) Performance vs. Regularization Parameter C

We sweep $C \in \{0.001, 0.01, 0.1, 1, 10, 100, 1000, 10000\}$.

Table 4: SVM performance for different C values

C	Train Acc	Train F1	Val Acc	Val F1	Test Acc	Test F1
0.001	0.2625	0.1040	0.2325	0.0943	0.2300	0.0935
0.01	0.3242	0.2182	0.3075	0.2249	0.3000	0.2111
0.1	0.8942	0.8959	0.8950	0.8949	0.9000	0.8987
1	0.9500	0.9505	0.9475	0.9463	0.9625	0.9617
10	0.9692	0.9696	0.9450	0.9442	0.9700	0.9693
100	0.9775	0.9778	0.9550	0.9543	0.9700	0.9696
1000	0.9867	0.9868	0.9500	0.9488	0.9700	0.9692
10000	0.9975	0.9975	0.9400	0.9389	0.9700	0.9697

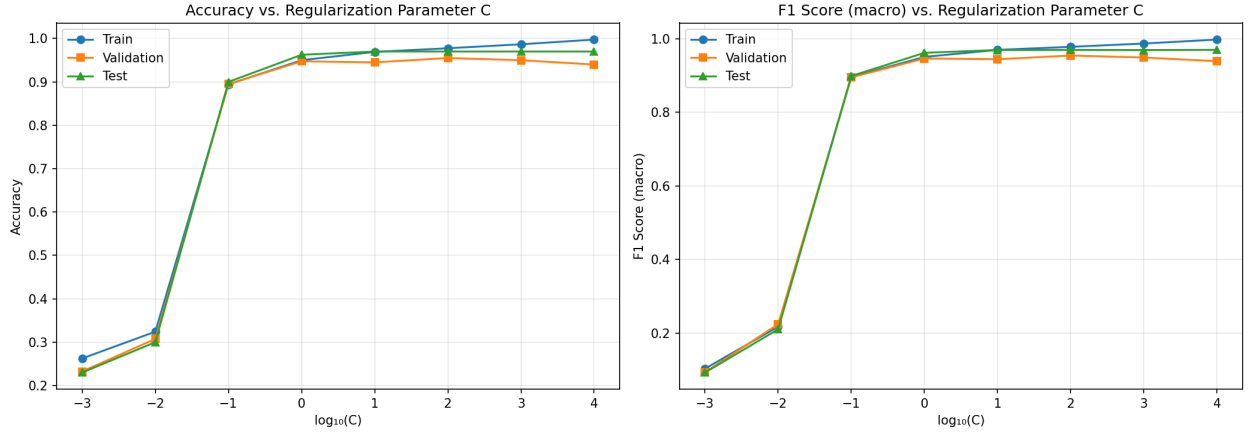


Figure 1: Accuracy and F1 score vs. $\log_{10}(C)$ for train, validation, and test sets.

2.3 2(c) Best Generalization Performance

Best $C = 100$ (selected by highest validation accuracy = 95.5%).

- **Underfitting at small C :** With $C \leq 0.01$, the SVM’s decision boundary is overly constrained, yielding accuracy near random chance (25% for 4 classes). The margin is too wide and misclassifies most samples.
- **Optimal range:** Performance improves dramatically from $C = 0.1$ ($\sim 89\%$) to $C = 1.0$ ($\sim 95\%$). Beyond $C = 1$, training accuracy continues climbing (reaching 99.8% at $C = 10000$) while validation accuracy peaks at $C = 100$ (95.5%) and then declines — a classic overfitting signal.
- **$C=100$ balances fit and generalization:** The train–validation gap is moderate (2.25%), and test performance is strong (97.0% accuracy, 97.0% F1). Although $C \in \{10, 1000, 10000\}$ achieve the same test accuracy, $C = 100$ has the best validation score, making it the most reliable choice without peeking at test data.

3 Q3: Association Rule Mining (15 pts)

We filter samples with `price_range == 1` (500 samples) and select 4 features: `ram`, `int_memory`, `px_width`, `battery_power`. Each feature is discretized into low/medium/high using a 3:4:3 **value range** split:

$$\text{low} \leq \min + 0.3 \times \text{range}, \quad \text{medium} \leq \min + 0.7 \times \text{range}, \quad \text{high} > \min + 0.7 \times \text{range}$$

The discretized features are converted to a boolean one-hot transaction matrix (500×12).

3.1 3(a) Frequent Patterns (support ≥ 0.3)

FP-Growth finds 8 frequent patterns:

Table 5: Frequent itemsets with support ≥ 0.3

Itemset	Support
{ram_medium}	0.682
{px_width_medium}	0.416
{battery_power_medium}	0.414
{int_memory_medium}	0.412
{battery_power_medium, ram_medium}	0.318
{int_memory_low}	0.316
{battery_power_low}	0.308
{ram_medium, px_width_medium}	0.306

3.2 3(b) Association Rules (support ≥ 0.3 , confidence ≥ 0.4 , lift ≥ 0.8)

Table 6: Association rules meeting all criteria

Antecedent	Consequent	Support	Confidence	Lift
ram_medium	battery_power_medium	0.318	0.466	1.126
battery_power_medium	ram_medium	0.318	0.768	1.126
ram_medium	px_width_medium	0.306	0.449	1.079
px_width_medium	ram_medium	0.306	0.736	1.079

3.3 3(c) Observations

- **ram_medium dominates:** 68.2% of `price_range=1` phones have mid-range RAM, making it the most frequent single item and the anchor of both 2-item frequent patterns. This aligns with the second-lowest price tier having predominantly mid-range specifications.
- **Asymmetric confidence:** `battery_power_medium` $\rightarrow$ `ram_medium` has 76.8% confidence, while the reverse has only 46.6%. This asymmetry arises because `ram_medium` has much higher base support (68.2%), so knowing a phone has medium battery power strongly predicts medium RAM, but not vice versa.

- **Mild positive associations:** All lift values are close to 1.0 (1.08–1.13), indicating that while medium-range features tend to co-occur slightly more than expected by chance, the associations are not dramatically strong. Within this price tier, the four features are somewhat independent.
- Only 2 two-item patterns and 4 rules pass all thresholds, reflecting the relatively uniform distribution of mid-range specifications in this price category.

4 Q4: PCA and K-Means Clustering (20 pts)

All 20 features from `mobile_price.csv` are standardized using z-score (`StandardScaler`). The target variable `price_range` (4 classes) is used only for evaluation, not for clustering.

4.1 4(a) Standardization

After `StandardScaler`, all features have mean ≈ 0 and std = 1.

4.2 4(b) PCA 2D Projection with True Labels

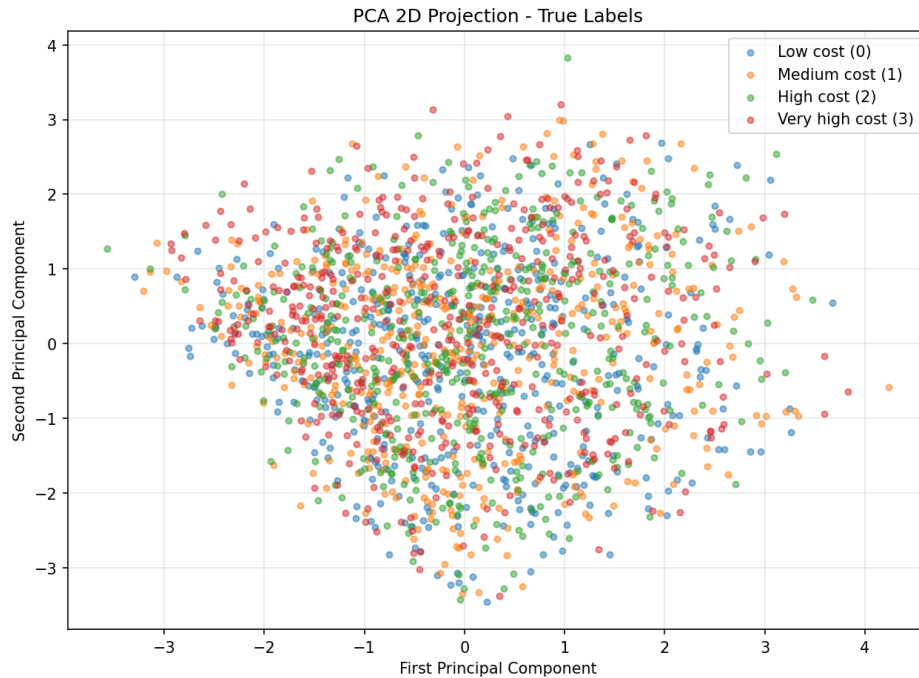


Figure 2: PCA 2D projection colored by true price-range labels. The first two PCs explain only 16.5% of total variance. Classes overlap heavily in this projection.

4.3 4(c) K-Means on All Standardized Features

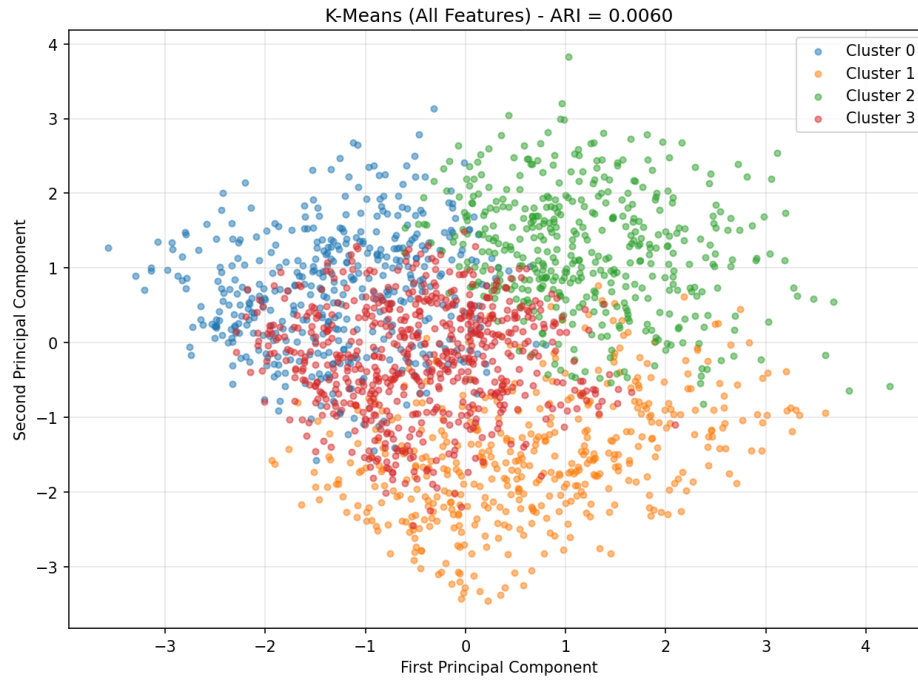


Figure 3: K-Means (all 20 features) visualized on PCA 2D plane. ARI = 0.0060.

4.4 4(d) K-Means on PCA 2D Features

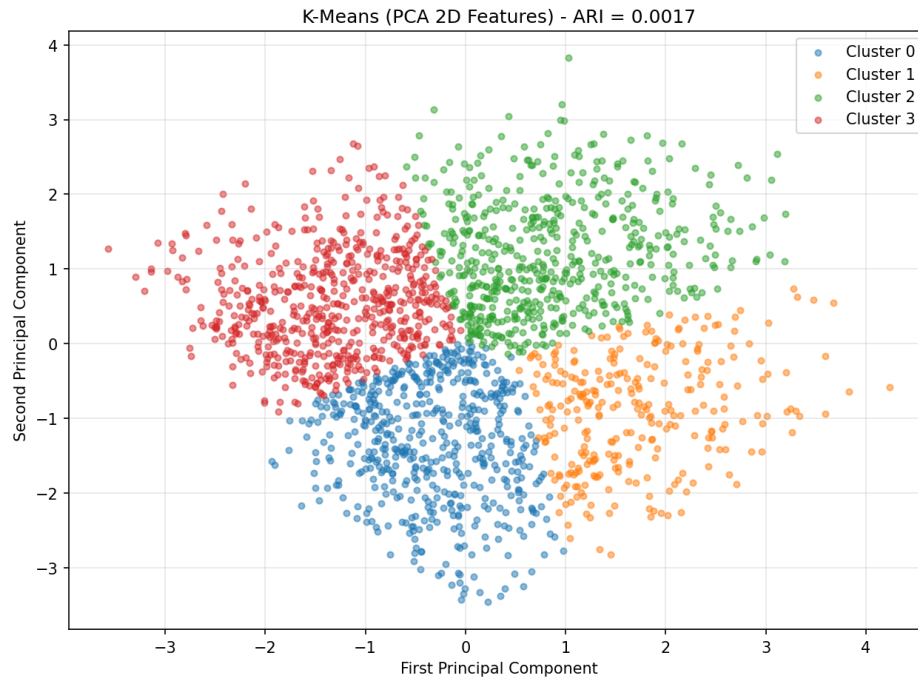


Figure 4: K-Means on PCA 2D features. ARI = 0.0017.

4.5 4(e) Observations

- **PCA captures very little variance:** The first two principal components explain only 16.5% of total variance. The 2D scatter plot shows heavy overlap among all 4 classes, indicating that the class structure is distributed across many dimensions rather than being concentrated in the top PCs.
- **Both K-Means approaches fail:** ARI values of 0.0060 (all features) and 0.0017 (PCA 2D) are near zero, meaning the clustering is essentially random with respect to the true labels.
- **Dimensionality reduction hurts slightly:** K-Means on all 20 features (ARI=0.0060) marginally outperforms K-Means on 2D PCA (ARI=0.0017), as expected since reducing to 2D discards 83.5% of the variance.
- **K-Means is fundamentally unsuited here:** K-Means assumes spherical, equally-sized clusters in Euclidean space. The mobile price classes likely have complex, non-linear decision boundaries. This contrasts sharply with Q2's SVM results (>95% accuracy), confirming that supervised methods with non-linear kernels (RBF) are necessary for this task.

5 Q5: Enhancing K-Means with Association Rule Mining (35 pts)

5.1 Motivation and Approach

Standard K-Means treats all features equally and ignores class-discriminative structure. We propose a two-pronged enhancement using association rule mining:

1. **Rule-Guided Feature Weighting:** All features are discretized (3:4:3 value range split) and association rules are mined per class using FP-Growth (`min_support=0.3`, confidence ≥ 0.5 , lift ≥ 1.0). Features that appear frequently in high-lift rules across classes receive higher weights. The resulting weights range from 1.0 to 3.0 and are applied to the standardized features before clustering.
2. **Class-Stratified Pattern Features:** The top 5 most discriminative patterns (ranked by lift $\times$ confidence) are selected from each class's rules, yielding 20 binary features. Each feature indicates whether a sample matches a specific class-associated itemset.

The augmented feature matrix combines 20 weighted original features with 20 scaled pattern features (40 total).

5.2 Evaluation Protocol

Both baseline and improved K-Means are evaluated using the same protocol:

- 5 random seeds: {0, 10, 42, 100, 999}
- **Hungarian algorithm** (`scipy.optimize.linear_sum_assignment`) maps cluster IDs to true labels optimally before computing accuracy, precision, recall, and F1 (all macro-averaged).

5.3 Results

Table 7: Per-seed and average comparison of baseline vs. improved K-Means

Seed	Method	Accuracy	Precision	Recall	F1
0	Baseline	0.2945	0.2940	0.2945	0.2926
	Improved	0.3885	0.4310	0.3885	0.3962
10	Baseline	0.2955	0.2953	0.2955	0.2936
	Improved	0.3905	0.4259	0.3905	0.3957
42	Baseline	0.2970	0.2966	0.2970	0.2950
	Improved	0.3885	0.4310	0.3885	0.3962
100	Baseline	0.2955	0.2950	0.2955	0.2935
	Improved	0.3885	0.4310	0.3885	0.3962
999	Baseline	0.2985	0.2984	0.2985	0.2956
	Improved	0.3885	0.4310	0.3885	0.3962
Average Baseline		0.2962	0.2959	0.2962	0.2941
Average Improved		0.3889	0.4300	0.3889	0.3961

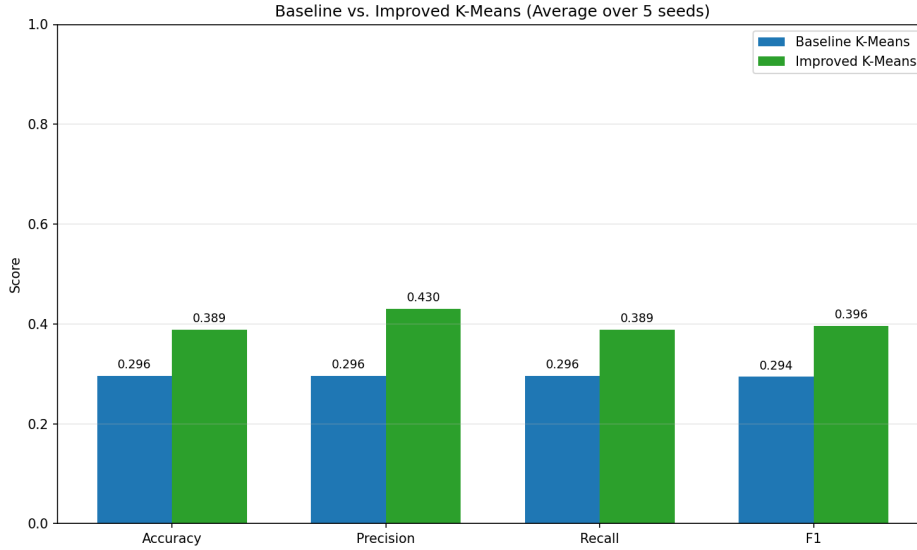


Figure 5: Average performance comparison across 5 seeds.

5.4 Analysis

- **Consistent improvement:** The improved method outperforms the baseline across all seeds and all metrics. Average accuracy increases from 29.6% to 38.9% (+9.3 pp), and F1 from 29.4% to 39.6% (+10.2 pp). Precision sees the largest relative gain (29.6% $\rightarrow$ 43.0%).
- **Feature weighting is key:** Association rule mining reveals that **ram** (weight=3.0), **three_g** (2.76), and **fc** (2.31) are the most class-discriminative features. Upweighting these features

helps K-Means focus on dimensions that actually separate the price classes, rather than treating noise features equally.

- **Pattern features add structural information:** The 20 binary pattern features encode class-specific co-occurrence patterns (e.g., {fc_low, ram_low, pc_low} for class 0; {ram_high, three_g_high, four_g_high} for class 3). These features inject discrete structural knowledge that continuous features alone cannot capture.
- **Limitations:** Even with enhancement, 38.9% accuracy remains far below the SVM's 97%. K-Means is inherently limited by its assumption of spherical clusters and its unsupervised nature. The improvement demonstrates that association rule mining can meaningfully guide clustering, but cannot replace the power of supervised learning with non-linear kernels for this dataset.
- **Stability:** The improved method shows very low variance across seeds (std=0.0009 for accuracy), compared to baseline (std=0.0016). The rule-guided features create more consistent cluster structure.